

GAMLSS: Location, Scale, and Shape Models

GAM.jl Contributors

Table of contents

Introduction	2
Setup	2
Normal Location-Scale Model	2
Data	3
Fitting the model	3
Extracting fitted parameters	3
Comparison with standard GAM	4
Visualisation	4
Constant-variance special case	5
Gamma Location-Scale Model	5
Data	6
Fitting	6
Results	6
Visualisation	7
Why not just <code>Gamma()</code> in <code>gam()</code> ?	7
Beta Regression	7
Data	8
Fitting	8
Results	8
Visualisation	9
Custom Links	9
Available Families	10
Effective Degrees of Freedom	11
R Comparison	11
Summary	13

Introduction

A standard GAM models the **mean** of a response distribution as a smooth function of covariates, keeping other distribution parameters (variance, shape) constant. **GAMLSS** (Generalized Additive Models for Location, Scale, and Shape) extends this by allowing *every* parameter of the response distribution to depend on covariates through smooth functions.

For a response y_i with distribution $\mathcal{D}(\theta_1, \theta_2, \dots, \theta_K)$, GAMLSS models each parameter via its own additive predictor:

$$g_k(\theta_{ki}) = \beta_{0k} + f_{k1}(x_{1i}) + \dots + f_{kJ_k}(x_{J_k i}), \quad k = 1, \dots, K$$

where g_k is the link function for parameter k . This is useful when:

- **Variance changes** with covariates (heteroscedasticity)
- **Skewness or tail behavior** varies across the covariate space
- You need a **distributional regression** rather than a mean regression

GAM.jl's `gamlss()` function provides this capability using Distributions.jl types and GLM.jl link functions.

Setup

```
using GAM
using CSV
using DataFrames
using Distributions
using GLM: LogLink, LogitLink, IdentityLink
using Statistics: mean, std, var, cor
using StatsAPI: fitted, deviance
using LinearAlgebra: diag
using Random
using Plots
```

Normal Location-Scale Model

The simplest GAMLSS: a Gaussian response where both the mean $\mu(x)$ and standard deviation $\sigma(x)$ vary smoothly with a covariate.

Data

We use data with:

- Location: $\mu(x) = 2 + 1.5 \sin(x)$
- Scale: $\sigma(x) = 0.3 + 0.2 \cos(x)$

```
dat = CSV.read("data_normal_ls.csv", DataFrame)
println("n = $(nrow(dat)), y range: [$(round(minimum(dat.y); digits=2)), $(round(maximum(dat.y); digits=2))]
```

```
n = 500, y range: [-0.33, 4.21]
```

Fitting the model

With `gamlss()`, pass `Normal()` directly — its parameters (μ, σ) match the `Distributions.jl` parameterization. Each parameter gets its own formula:

```
m = gamlss(
    [@gam_formula(y ~ s(x, k=15, bs=:cr)), # formula
     @gam_formula(y ~ s(x, k=10, bs=:cr))], # log() formula
    dat, Normal()
)
println("Converged: $(m.converged)")
println("Negative log-likelihood: $(round(m.nll; digits=2))")
println("Total EDF: $(round(sum(m.edf); digits=1))")
```

```
Converged: true
Negative log-likelihood: 25.99
Total EDF: 18.6
```

The default links are `IdentityLink()` for μ and `LogLink()` for σ , matching the standard GAMLSS parameterization.

Extracting fitted parameters

The fitted linear predictors are stored in `m.fitted_eta`. Apply the inverse link to recover the distribution parameters:

```

_fit = m.fitted_eta[1]          # identity link → directly
_fit = exp.(m.fitted_eta[2])    # log link → exp to get

println(" : cor with truth = $(round(cor(_fit, dat.mu_true); digits=5))")
println(" : cor with truth = $(round(cor(_fit, dat.sigma_true); digits=5))")

```

```

: cor with truth = 0.99861
: cor with truth = 0.98639

```

Comparison with standard GAM

A standard `gam()` assumes constant variance. Let's compare:

```

m_gam = gam(@gam_formula(y ~ s(x, k=15, bs=:cr)), dat)
_gam = m_gam.fitted_values
println("Standard GAM : cor = $(round(cor(_gam, dat.mu_true); digits=5))")
println("GAMLSS : cor = $(round(cor(_fit, dat.mu_true); digits=5))")
println("Max |_gam - _gamlss|: $(round(maximum(abs.(_gam - _fit)); digits=4))")

```

```

Standard GAM : cor = 0.99873
GAMLSS : cor = 0.99861
Max |_gam - _gamlss|: 0.0357

```

The location estimates are similar, but GAMLSS additionally captures how the spread changes.

Visualisation

```

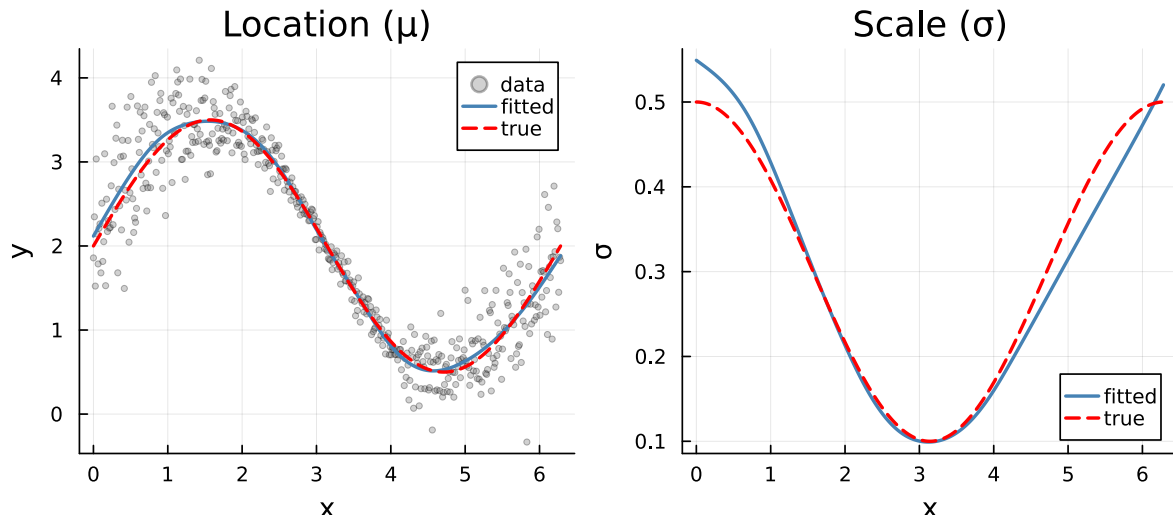
idx = sortperm(dat.x)

p1 = scatter(dat.x, dat.y; label="data", ms=2, alpha=0.3, color=:grey40,
             xlabel="x", ylabel="y", title="Location ( )")
plot!(p1, dat.x[idx], _fit[idx]; label="fitted", lw=2, color=:steelblue)
plot!(p1, dat.x[idx], dat.mu_true[idx]; label="true", lw=2, ls=:dash, color=:red)

p2 = plot(dat.x[idx], _fit[idx]; label="fitted", lw=2, color=:steelblue,
          xlabel="x", ylabel=" ", title="Scale ( )")
plot!(p2, dat.x[idx], dat.sigma_true[idx]; label="true", lw=2, ls=:dash, color=:red)

```

```
plot(p1, p2; layout=(1, 2), size=(700, 300))
```



Constant-variance special case

When σ is modeled as intercept-only ($y \sim 1$), GAMLSS reduces to a standard GAM:

```
m_const = gamlss(
  [@gam_formula(y ~ s(x, k=15, bs=:cr)),
  @gam_formula(y ~ 1)],
  dat, Normal()
)
_const = exp.(m_const.fitted_eta[2])
println(" range: [$(round(minimum(_const); digits=4)), $(round(maximum(_const); digits=4))])
println(" CV: $(round(std(_const)/mean(_const)*100; digits=2))%")
```

```
range: [0.3294, 0.3294]
CV: 0.0%
```

Gamma Location-Scale Model

For positive continuous data, `GammaLocationScale()` reparameterizes the Gamma distribution as:

- $\mu = \text{mean}(\text{link: LogLink}())$

- σ = coefficient of variation (link: `LogLink()`)

This maps to `Gamma( = 1/  $\sigma^2$ , =  $\mu^2$ )` so that $E[Y] = \mu$ and $\text{Var}[Y] = \mu^2 \sigma^2$.

Data

```
dat_g = CSV.read("data_gamma_ls.csv", DataFrame)
println("n = $(nrow(dat_g)), y range: [$(round(minimum(dat_g.y); digits=3)), $(round(maximum(dat_g.y); digits=3)))]")
println("All positive: $(all(dat_g.y .> 0))")
```

```
n = 500, y range: [0.563, 7.16]
All positive: true
```

Fitting

```
m_g = gamlss(
  [@gam_formula(y ~ s(x, k=15, bs=:cr)), # log( )
  @gam_formula(y ~ s(x, k=10, bs=:cr))], # log( )
  dat_g, GammaLocationScale()
)
println("Converged: $(m_g.converged)")
```

```
Converged: true
```

Results

```
_fit = exp.(m_g.fitted_eta[1]) # log link for
_fit = exp.(m_g.fitted_eta[2]) # log link for (CV)

println(": cor with truth = $(round(cor(_fit, dat_g.mu_true); digits=5))")
println(": cor with truth = $(round(cor(_fit, dat_g.sigma_true); digits=5))")
println("Mean fitted CV: $(round(mean(_fit); digits=3))")
```

```
: cor with truth = 0.83194
: cor with truth = 0.99327
Mean fitted CV: 0.368
```

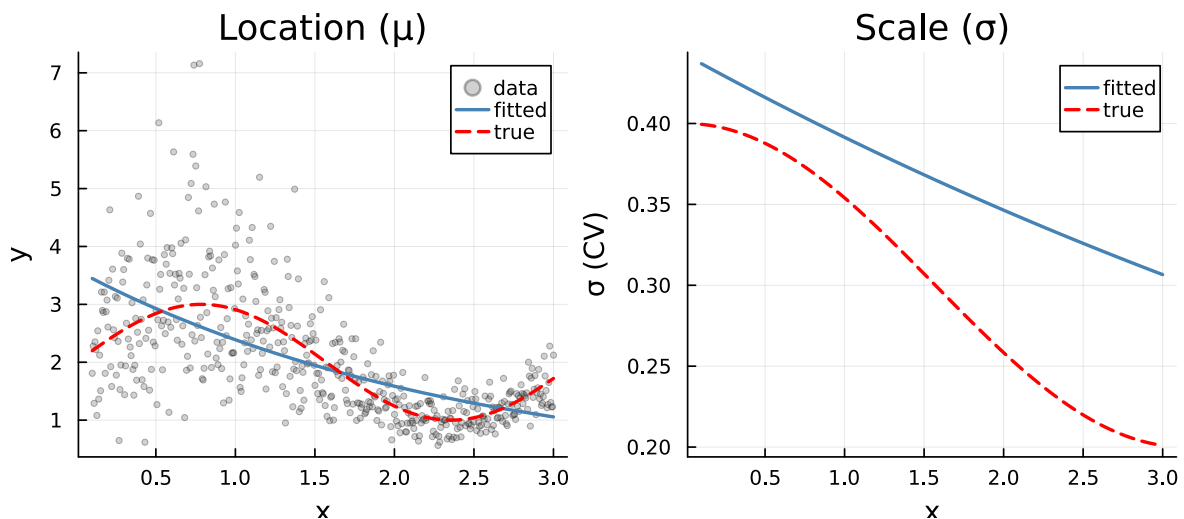
Visualisation

```
idx_g = sortperm(dat_g.x)

p1 = scatter(dat_g.x, dat_g.y; label="data", ms=2, alpha=0.3, color=:grey40,
             xlabel="x", ylabel="y", title="Location ( )")
plot!(p1, dat_g.x[idx_g], _fit[idx_g]; label="fitted", lw=2, color=:steelblue)
plot!(p1, dat_g.x[idx_g], dat_g.mu_true[idx_g]; label="true", lw=2, ls=:dash, color=:red)

p2 = plot(dat_g.x[idx_g], _fit[idx_g]; label="fitted", lw=2, color=:steelblue,
          xlabel="x", ylabel=" (CV)", title="Scale ( )")
plot!(p2, dat_g.x[idx_g], dat_g.sigma_true[idx_g]; label="true", lw=2, ls=:dash, color=:red)

plot(p1, p2; layout=(1, 2), size=(700, 300))
```



Why not just `Gamma()` in `gam()`?

A standard `gam(...; family=Gamma())` only models the mean with a single scale parameter. The GAMLSS version lets the CV *vary* with x , capturing heterogeneity in the dispersion.

Beta Regression

For responses bounded in $(0,1)$ — proportions, rates, or compositional data — `BetaRegression()` reparameterizes the Beta distribution as:

- μ = mean (link: LogitLink())
- ϕ = precision (link: LogLink())

This maps to $\text{Beta}(\mu = \mu\phi, \phi = (1-\mu)\phi)$ so that $E[Y] = \mu$ and $\text{Var}[Y] = \mu(1 - \mu)/(1 + \phi)$.

Data

```
dat_b = CSV.read("data_beta_reg.csv", DataFrame)
println("n = $(nrow(dat_b)), y range: [$(round(minimum(dat_b.y); digits=4)), $(round(maximum(dat_b.y); digits=4)))]")
```

```
n = 400, y range: [0.0071, 0.7526]
```

Fitting

Here ϕ is constant (intercept-only), so we model only the mean as smooth:

```
m_b = gamlss(
  [@gam_formula(y ~ s(x, k=15, bs=:cr)), # logit()
  @gam_formula(y ~ 1)],               # log()
  dat_b, BetaRegression()
)
println("Converged: $(m_b.converged)")
```

```
Converged: true
```

Results

```
logit_inv(x) = 1 / (1 + exp(-x))
_fit = logit_inv(m_b.fitted_eta[1])
_fit = exp(m_b.fitted_eta[2])

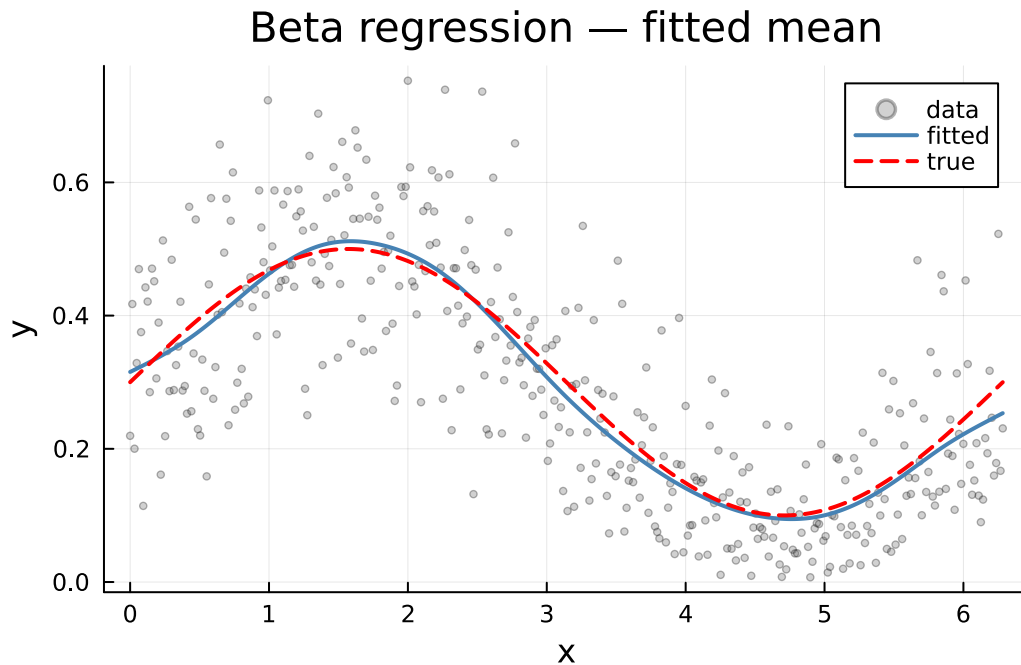
println(": cor with truth = $(round(cor(_fit, dat_b.mu_true); digits=5))")
println("Fitted precision    $(round(mean(_fit); digits=1)) (true: 15.0)")
```

```
: cor with truth = 0.99727
Fitted precision    16.5 (true: 15.0)
```


Visualisation

```
idx_b = sortperm(dat_b.x)

scatter(dat_b.x, dat_b.y; label="data", ms=2, alpha=0.3, color=:grey40,
        xlabel="x", ylabel="y", title="Beta regression - fitted mean")
plot!(dat_b.x[idx_b], _fit[idx_b]; label="fitted", lw=2, color=:steelblue)
plot!(dat_b.x[idx_b], dat_b.mu_true[idx_b]; label="true", lw=2, ls=:dash, color=:red)
```



Custom Links

Links are specified separately from the family, just like in GLM.jl. For `Normal()`, pass them via the `links` keyword:

```
# Use log link for both x and y (e.g., when y must be positive)
Random.seed!(123)
x_pos = collect(range(0.1, 3.0; length=200))
_pos = exp.(0.5 .+ 0.3 .* sin.(2 .* x_pos))
y_pos = _pos .+ 0.2 .* randn(200)
y_pos = max.(y_pos, 0.01)
dat_pos = DataFrame(x=x_pos, y=y_pos)
```

```

m_custom = gamlss(
  [@gam_formula(y ~ s(x, k=10, bs=:cr)),
   @gam_formula(y ~ 1)],
  dat_pos, Normal();
  links=[LogLink(), LogLink()] # both parameters on log scale
)
println("Converged: $(m_custom.converged)")
println("Fitted range: [$(round(minimum(exp.(m_custom.fitted_eta[1])); digits=2)), $(round(

```

```

Converged: true
Fitted range: [1.26, 2.17]

```

For reparameterized families, pass links to the constructor:

```

m_g2 = gamlss(
  [@gam_formula(y ~ s(x, k=10, bs=:cr)),
   @gam_formula(y ~ 1)],
  dat_g, GammaLocationScale(links=[LogLink(), LogLink()])
)
println("Converged: $(m_g2.converged)")

```

```

Converged: true

```

Available Families

GAM.jl's `gamlss()` supports several distribution families:

Family	Parameters	Default links	Type
Normal()	(mean), (SD)	Identity, Log	Direct Distributions.jl
GammaLocationScale()	(mean), (CV)	Log, Log	Reparameterized
BetaRegression()	(mean), (precision)	Logit, Log	Reparameterized
NegativeBinomialLocationScale()	(mean), (precision), (overdispersion)	Log, Log	Reparameterized
InverseGaussianLocationScale()	(mean), (CV)	Log, Log	Reparameterized

Family	Parameters	Default links	Type
GEVFamily()	, ,	Identity, Log, Identity	Extreme value
GPDFamily()	,	Log, Identity	Extreme value

Legacy aliases (`GaussianLS()`, `GammaLS()`, `BetaLS()`, `NegBinLS()`) are also available for backward compatibility.

Effective Degrees of Freedom

Each smooth in each parameter equation has its own EDF, estimated via REML:

```
println("Normal location-scale model:")
K = GAM.nparams(m.family)
offsets = m.param_offsets
for k in 1:K
    s = offsets[k] + 1
    e = offsets[k + 1]
    edf_k = sum(m.edf[s:e])
    pname = GAM.param_names(m.family)[k]
    println("  $(pname): total EDF = $(round(edf_k; digits=2)) ($(e-s+1) coefficients)")
end
```

```
Normal location-scale model:
mu: total EDF = 11.51 (15 coefficients)
sigma: total EDF = 7.07 (10 coefficients)
```

R Comparison

The R `gamlss` package (with `gamlss.add` for smooth terms) fits similar models. Here we verify `GAM.jl` matches R:

```
using RCall

# Re-fit the Normal location-scale model for comparison
m_norm = gamlss(
    [@gam_formula(y ~ s(x, k=15, bs=:cr)),
     @gam_formula(y ~ s(x, k=10, bs=:cr))],
```

```

    dat, Normal())
_fit_norm = m_norm.fitted_eta[1]
_fit_norm = exp.(m_norm.fitted_eta[2])

R"""
suppressPackageStartupMessages({
  library(mgcv)
  library(gamlss)
  library(gamlss.add)
})
"""

# Pass data to R and fit Normal location-scale
@rput dat
R"""
m_r <- gamlss(y ~ ga(~s(x, k=15, bs="cr")),
             sigma.formula = ~ ga(~s(x, k=10, bs="cr")),
             family = NO(), data = dat, trace = FALSE,
             control = gamlss.control(n.cyc = 100, trace = FALSE))
mu_r <- fitted(m_r, "mu")
sigma_r <- fitted(m_r, "sigma")
"""

@rget mu_r sigma_r

n_julia = length(_fit_norm)
n_r = length(mu_r)
if n_julia == n_r
  println(" : Julia-R correlation = $(round(cor(_fit_norm, mu_r); digits=6))")
  println(" : Julia-R correlation = $(round(cor(_fit_norm, sigma_r); digits=6))")
  println(" : max |diff| = $(round(maximum(abs.(_fit_norm - mu_r)); digits=4))")
  println(" : max |diff| = $(round(maximum(abs.(_fit_norm - sigma_r)); digits=4))")
else
  println("Length mismatch: Julia=$n_julia, R=$n_r - comparing first $n_r")
  _j = _fit_norm[1:n_r]
  _j = _fit_norm[1:n_r]
  println(" : Julia-R correlation = $(round(cor(_j, mu_r); digits=6))")
  println(" : Julia-R correlation = $(round(cor(_j, sigma_r); digits=6))")
end

```

Summary

- `gamlss()` fits distributional regression models where each parameter gets its own smooth predictor
- Pass **Distributions.jl types** directly (`Normal()`) when parameters match, or use **reparameterized types** (`GammaLocationScale()`, `BetaRegression()`) when they don't
- **Links** are always separate — via `links=` keyword for direct types, or in the family constructor for reparameterized types
- An intercept-only scale formula (`y ~ 1`) recovers the standard GAM as a special case
- Smoothing parameters are estimated jointly via **REML**
- Results can be compared against R's `gamlss` package with `gamlss.add` smooth terms